

TECHNICAL DESIGN DOCUMENT

REVISION NOTES

v1.6 to v1.7

A High-Performance Reverse Proxy
& Load Balancer in Go

Berk Egemen Oğuz

Prepared: 10 September 2026

Version: 1.7

Contents

0. Cover and version	3
1. Section 3.1 — Folder and package layout	3
2. Section 3.3 — Configuration schema	3
3. Section 5.3 — Weighted round robin	4
4. Section 5.5 — Backend failure behaviour	5
5. Section 6.2 — Example interface design	5
6. Sections 7.1, 7.3 and 11 — Branching and pull requests	7
7. Section 7.6 — Monitoring after deployment	7
8. Section 8.1 — Example docker-compose.yml	8
9. Section 9 — The twelve day plan table	8
10. Section 10.3 — Load testing	9
11. Section 14 — Conclusion and next steps	9
12. Suggested addition: a new section	10

Written after the twelve day plan was finished, to bring the design document in line with the system that was actually built. Each entry gives the **current text** and the **replacement text**, so only the affected paragraphs change and the document keeps its figures, tables and formatting.

The design document itself is in Turkish. The replacement text here is in English, for a reader who does not read Turkish or for an English edition of the document; the text to paste into the Turkish document is in `revision-notes-v1.7-tr.pdf`.

The reasoning behind each change is in `docs/design-deviations.md` in the repository, referenced entry by entry.

0. Cover and version

Current: `Sürüm: 1.1` on the cover, with the content referred to as v1.6.

New: `Version: 1.7`, with the date updated. Add to the cover:

This revision was made after the twelve day implementation plan was completed, and describes the system as it was actually built. Where a design decision turned out differently in practice, that is stated.

1. Section 3.1 — Folder and package layout

Current: `internal/` lists `config`, `balancer`, `health`, `proxy`, `server`, `observability`.

New: add two packages:

```
|  └─ app/                # wiring that assembles the modules
|  └─ integration/        # end-to-end tests
```

Why: had the wiring stayed in `main.go`, the integration tests would have assembled their own copy of the chain and verified something the binary does not use. `cmd/lb/main.go` now only reads the configuration, builds the logger and calls `app.New`.

2. Section 3.3 — Configuration schema

Three fields were added. Update the example YAML.

Directly below `listen_addr`:

```
metrics_addr: ":8081"      # /metrics and /status
enable_pprof: false       # adds /debug/pprof to the metrics port
```

In the `timeouts` block, below `connect_timeout`:

```
response_timeout: 10s      # abandon and retry a backend that does not start answering
```

Add to the explanatory bullets:

metrics_addr — the observability endpoints are served on a socket of their own, separate from proxied traffic. On the traffic port, `/metrics` and `/status` could never be proxied to a backend, and internal state would be exposed to anyone who can reach the balancer. That server also carries no connection limit: observability has to keep working exactly when the traffic port is saturated.

enable_pprof — adds Go's profiling endpoints. Off by default, because they expose heap and goroutine state.

response_timeout — bounds how long a backend may take to start answering once it has accepted the connection. Defaults to `read_timeout` when omitted. Without it, a stalled backend holds the request until the client-side `write_timeout` kills it and the failure policy never gets a chance, which is what the slow-backend scenario in section 10.4 asks about.

See deviations 5 and 6.

3. Section 5.3 — Weighted round robin

Current:

Each backend is given a weight in the configuration; selection is probabilistic, in proportion to those weights.

New:

Each backend is given a weight in the configuration, and selection is deterministic, using the smooth weighted round robin algorithm: every selection raises each backend's credit by its weight, the backend with the most credit serves the request, and its credit then drops by the total weight.

Compared with picking a backend at random in proportion to its weight, this hits the configured ratio exactly rather than approximately, and spreads the heavy backend's turns across the cycle: with weights 5, 1 and 1 the sequence is `a a b a c a a`. Needing no random source also makes the tests exact rather than statistical: with weights 1, 2 and 3, twelve hundred requests divide into exactly 200, 400 and 600.

See deviation 2.

4. Section 5.5 — Backend failure behaviour

Add a further item to the list of HTTP responses by failure scenario:

Health checking has marked every backend unhealthy — the request is offered to the pool anyway and forwarded to a backend that has not been tried, and the fallback is recorded under a separate counter (`no_healthy_backend`). Under load the health probes time out before client traffic does, and every backend can be marked unhealthy at once; refusing all traffic then turns a slow system into a broken one. A backend that may still answer is worth one attempt, and a backend that is genuinely dead costs one failed attempt before the client receives the 503 it would have received anyway.

The observable consequence: when every backend is unhealthy but still answering, the client receives the backend's own response rather than a 503 from the balancer.

This was not hypothetical: at saturation during the load test, 275,769 requests were refused. See deviation 4.

5. Section 6.2 — Example interface design

Current:

```
type Backend struct {
    Addr    string
    Weight  int
    Healthy bool
}
```

```
// internal/health package
type Checker interface {
    Start(ctx context.Context, backends []*Backend)
    IsHealthy(addr string) bool
}
```

New:

```
// internal/balancer package
type Backend struct {
    Addr    string
    Weight  int

    // active is the number of requests this backend is serving right now,
    // which is what least connections balances on. Updated atomically.
    active atomic.Int64
}

// internal/health package
type Checker interface {
    Start(ctx context.Context, backends []*Backend)
    IsHealthy(addr string) bool
    ReportSuccess(addr string)
    ReportFailure(addr string)
    Reload(ctx context.Context, cfg config.HealthCheck, backends []*Backend)
}
```

Add:

Health is not a field on `Backend` but state held inside the checker, keyed by address, so that active and passive checking feed the same counters and the state survives a rebuild of the pool.

`ReportSuccess` and `ReportFailure` are what passive health checking from section 6.1 needs: the proxy reports the outcome of every request it forwards, so a backend failing real traffic leaves the pool without waiting for the next probe. `Reload` switches the backend set when the configuration is reloaded.

See deviation 3.

6. Sections 7.1, 7.3 and 11 — Branching and pull requests

The largest difference in the document. Replace the branching strategy in 7.1 and the first item of 7.3 with:

Branching strategy. The project is carried out by a single developer, so all work is committed straight to `main`; feature branches and the pull request flow are not used. With no second person to review, a pull request adds process without acting as a quality gate.

CI is the gate instead: every push to `main` runs `gofmt`, `go vet`, `golangci-lint`, `govulncheck`, the build and the full test suite under `-race`. If the team grows, the protected branch and pull request flow described here can be turned back on; the pipeline already runs the checks a pull request would need.

The `experiment/epoll-loop` branch (section 4, phase 2) and the `release/vX.Y.Z` tagging approach are unchanged.

In section 11, the version control criterion:

Version control: every change has passed CI; the `v1.0.0` tag is published in GitHub Releases and the corresponding container image is pushed to the registry.

See deviation 1.

7. Section 7.6 — Monitoring after deployment

Current: the Grafana service has `mem_limit: 256m`.

New: `mem_limit: 512m`, with a note:

Grafana 13 stays within 256 MB while idle but exceeds it while rendering a dashboard, and the container is killed by the out-of-memory handler (exit 137). Measured under load it uses 331 MiB, so its limit is 512 MB. Prometheus uses 143 MiB while scraping ten backends and keeps its 256 MB limit.

Two smaller corrections:

- The Prometheus volume is written as `./deploy/prometheus.yml:/etc/prometheus/prometheus.yml`; since the compose file lives in `deploy/`, it should be `./prometheus.yml:/etc/prometheus/prometheus.yml`.

- Only the compose service should be listed as a scrape target. Scraping `loadbalancer:8081` and `host.docker.internal:8081` at the same time reaches the same process, because the compose service publishes the metrics port on the host, and every aggregate query then reports double.

See deviation 7.

8. Section 8.1 — Example docker-compose.yml

Add the load balancer service to the example:

```
loadbalancer:
  build:
    context: ..
    args:
      VERSION: dev
  ports: ["8080:8080", "8081:8081"]
  volumes: ["../configs/lb.example.yaml:/etc/lb/config.yaml:ro"]
  depends_on: [backend-1, backend-2]
  mem_limit: 128m
```

With a note:

The image is built on a distroless base and carries no shell, so no container healthcheck is defined; `/status` on the metrics port serves that purpose from outside.

9. Section 9 — The twelve day plan table

Day 5, current: "Active connection counter, weighted probabilistic selection, unit tests."

New: "Active connection counter, deterministic weighted selection using smooth weighted round robin, unit tests."

Day 11: the deliverable column can be updated to "A system whose resilience is verified, whose configuration reloads on SIGHUP, and which is documented."

10. Section 10.3 — Load testing

Current: "Throughput and p50/p95/p99 latency are measured with wrk or ab (Apache Bench) at rising concurrency levels (for example 100, 1,000 and 10,000 connections)."

New:

Reference measurements are taken with `ab` (Apache Bench). `ab` is single threaded, however, and cannot measure beyond a thousand connections, where it fails with `apr_socket_recv: Operation timed out`. Higher levels are driven by a small purpose-built tool: one goroutine per connection, keep-alive throughout, latency percentiles from recorded samples. The backends are loaded directly first, to establish whether a measurement describes the balancer or the generator.

Replace the target bullets with the measured results:

Measured on a single ten core machine that was also running the load generator and all ten backends: 40,616 requests per second at 100 connections, with p50 2.1 ms, p95 5.3 ms and p99 7.7 ms; 41,208 per second at 1,000 connections, with p95 46.7 ms; 41,421 per second at 2,000 connections, with p95 87.5 ms. No request failed at any level.

The p95 target should be stated together with the load level it applies to. At 100 concurrent connections it is met; the measurement at 1,000 connections was taken with the balancer sharing a machine with the load generator and the backends, which is not representative of a deployment.

See deviations 8 and 9.

11. Section 14 — Conclusion and next steps

Update the opening paragraph:

The plan described in this document was completed in twelve days, tagged v1.0.0, and produced a container image that can be deployed to production. Three load balancing algorithms, active and passive health checking, configurable failure policies, rate limiting and resource limits, structured logging with Prometheus-based observability, and configuration reload on SIGHUP have all been implemented. Unit test coverage in the critical packages is between 89% and 100%, with 22 integration tests running over real sockets.

12. Suggested addition: a new section

Consider adding a section titled "15. Post-implementation review". It can be assembled from three documents in the repository:

- the load test results and the three bottlenecks found — `docs/performance-report.md`
- the deviations from the design and their reasoning — `docs/design-deviations.md`
- how the production readiness criteria were met — `docs/deployment-checklist.md`

That turns the document from a plan into a design reference that also records how the plan turned out, which is what section 1 says it is for.